Supplemental Material

Coverage, Not Freshness: Authorized Vector Search under Policy Churn

Yibin Chen, Xiaofei Di, Haiming Liu, Haiyang Zhen, Yige Yan, and Xiaoqing Li
Beijing Jiaotong University

***Abstract*—This supplement carries the figures, tables and measurement notes that the 12-page main text summarises. Nothing here is required to follow the argument; every item is either a detail of a number quoted in the text or a reproduction note.**

I. What moved here

- The architecture figure (SII) and the lease timeline (SIV), which illustrate mechanics the text describes in words.
- The full-size evaluation figure (SV), the per-role HNSW table (SVI), the three-laws table (SVII) and the dead-fraction grid (SVIII).
- The baseline protocol (SX), the HoneyBee run, and the measurement-variance record (SXI).
- The mechanism ablation and deployment boundary (SXXIII), with the five-arm table and the p95 condition.

II. Architecture

LeaseServe has three planes: a read plane that pins a version, checks the certificate and takes candidates from the graph and the log; a control plane that consumes changes, keeps the lease table and schedules merges; and an exact plane that serves small roles and every certified failure. The figure is the picture of the text.

III. Control-plane algorithm

Algorithm S1 is the publication and merge path referred to in §III of the main text.

Algorithm S1: Publication and merge of a lease (control plane)

procedure *Grant*(r, d)
  $e \leftarrow$ NextEpoch()
  append (e, d) to D_r ▷ `log first`
  $\omega_r \leftarrow e$ ▷ `then the watermark`
  Publish(e) ▷ `readers may now pin` e
procedure *Revoke*(r, d)
  $e \leftarrow$ NextEpoch()
  $r_d \leftarrow e$ ▷ `filter only: no index work`
  $\omega_r \leftarrow e$ ▷ `the log still covers every grant` $\leq e$
  Publish(e)
procedure *Merge*(r)
  $a' \leftarrow$ CurrentEpoch() ▷ `built off the read path`
  $G' \leftarrow$ a graph built over $V(r, a')$
  Lock(r) ▷ `the single writer serializes publication`
  $\omega' \leftarrow$ CurrentEpoch(); $\Delta_{\text{suf}} \leftarrow$ log entries with epoch in $(a', \omega']$
  PublishLease$(r, a', \omega', G', \Delta_{\text{suf}})$ ▷ `suffix replayed, one pointer store`
  Unlock(r)
  retain L until every live pin is $\geq a'$
  reclaim G_r and the folded log entries below the minimum live pin

IV. Lease timeline

One lease across six events: a build at epoch 5, a grant at 10, revocations at 12 and 14, a merge at 20, and the reader that pinned 10 before the merge and keeps the old lease until reclamation.

V. The evaluation figure at full size

Fig. 3 is the main paper's evaluation figure at full width, built by the same script from the same sources (`paper-v2/figs/src/fig_results.py`); each panel names its source table or run directory in the script header.

VI. Per-role HNSW lease

VII. Three measured laws

Replaying one change stream under all four designs on three ACL structures gives the totals below. The totals are 2,988/2,990/2,786; the authorization components are 2,988/2,980/2,786, within 7%, while the four designs' totals on the same queries range from 0 to 16,288.

VIII. Dead-fraction grid

Missing top-10 results per read as the dead fraction grows at four candidate multipliers. The mean criterion

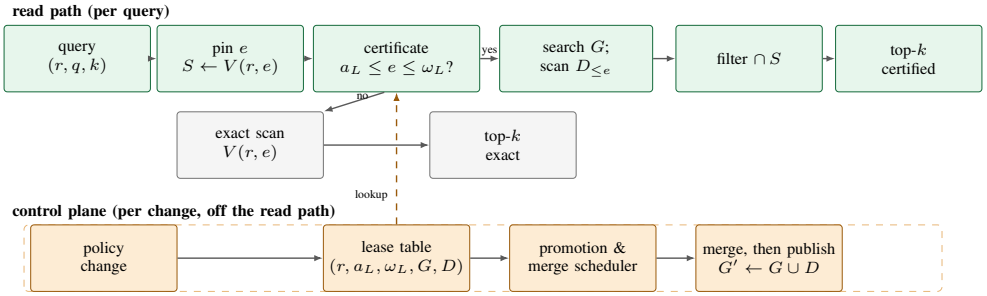


Fig. 1. LeaseServe. The read path never repairs: it pins an epoch, checks the certificate, and takes candidates from the graph and the log; a failure of the certificate falls back to the exact scan of the pinned snapshot. The control plane consumes changes into the lease table and schedules merges; a merge publishes a new lease with one pointer store and never blocks a reader.

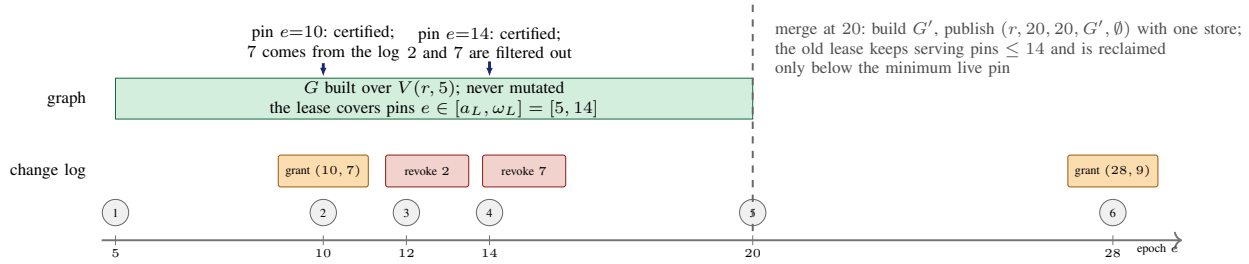


Fig. 2. A lease across six events. The graph is built once at epoch 5 and never mutated; a grant appends to the log, a revocation only marks the document; ω_L advances at every change, so the lease is usable for every pin in $[5, 14]$. A merge at epoch 20 publishes a new lease with one pointer store; a read that pinned 10 before the merge keeps the old lease, which still covers its pin.

TABLE I

C++ PER-ROLE HNSW LEASE, 1M SIFT, K=10 HEAD ROLES, 200 CHANGES. BUILD IS THE TEN HEAD LEASES TOGETHER; UPDATE IS THE TOTAL FOR THE 200-CHANGE STREAM (SINGLE-SESSION PROTOCOL); MISSING OVER 5000 TOP-10 RESULTS; LEAKAGE IS ZERO IN EVERY RUN. ERBAC ROWS ARE THE CONTRACT BUILD WITH FIVE SEEDS ON A QUIET AMD 9950X (SD AT MOST 0.2 S, 0.008 MS AND 0.6 FOR MISSING); UNIFORM ROWS ARE THE LOCKED BUILD, SINGLE RUNS FROM THE EARLIER SESSION, WHOSE CONCURRENT REPEATS READ 0.27/0.67 (UNIFORM) AND 0.60/3.7 (ERBAC) ON A LOADED HOST, AND 0.16/0.51 AND 0.35/1.8 ON A QUIET XEON GOLD 6330, SO ABSOLUTE CONCURRENT P95 IS HOST-DEPENDENT.

Data / mode	build (s)	update (s)	memory	p50	p95	missing
Uniform, sequential	3.75	0.020	1.05×	0.216	0.528	0
Uniform, 4R+1W	3.90	0.020	1.05×	0.405	0.939	5
ERBAC, sequential	40.1	0.236	1.77×	0.139	0.167	6
ERBAC, 4R+1W	39.1	0.236	1.77×	**0.184**	0.228	0.6

TABLE II

MISSING TOP-10 RESULTS UNDER ONE CHANGE STREAM, FOUR DESIGNS, THREE ACL STRUCTURES (1M VECTORS, 200 EVENTS, FIVE SEEDS; 18,000 SCORED RESULTS PER ACL). THE RANKING OF DESIGNS FLIPS WITH SELECTIVITY WHILE THE STATIC COLUMN BARELY MOVES (2,988/2,990/2,786): ON ERBAC IT DECOMPOSES AS 2,980 AUTH PLUS 10 ANN, THE COMPONENT THE MAIN TEXT QUOTES AS THE AUTH TERM.

ACL (selectivity)	lease	static	partition	shared
Uniform (0.5%)	**0**	2,988	7,421	16,288
ERBAC (7–10%)	17	**2,990**	103	1,093
OpenAlex (1.6–3.8%, real)	2	2,786	179	10,076

TABLE III

DEAD FRACTION AGAINST CANDIDATE MULTIPLIER (MISSING TOP-10 RESULTS PER READ). THE MEAN CRITERION PREDICTS LOSS AT ρ=1 FOR EVERY ROW; AT ρ=2 THE LOSS IS FLAT RATHER THAN ZERO, WHICH IS THE TAIL THE MEAN DOES NOT PRICE.

φ	ρ=1	ρ=2	ρ=5	ρ=15
0.02	1.13	0.35	0.025	**0**
0.10	1.35	0.36	0.027	**0**
0.26	1.93	0.37	0.031	**0**
0.45	2.61	0.38	0.033	**0**

predicts loss at ρ=1 for every row; at ρ=2 the loss is flat rather than zero, which is the tail the mean does not price.

IX. REPAIR VERSUS REBUILD

PRO-HNSW (ICDE 2026) is a repair vocabulary for HNSW graphs that lose data physically: it prunes edges into deleted nodes (ROE), reconnects nodes stranded in small components (RDN) and restores reciprocity (RUE), cheaply after every batch and deeply once at the end. A lease can use only the second half of that setting. Its published graph serves every pin from the build to the last covered grant, so a document revoked now may still be a live candidate for an older pin and may not be deleted; what remains is to rewire the graph at a version

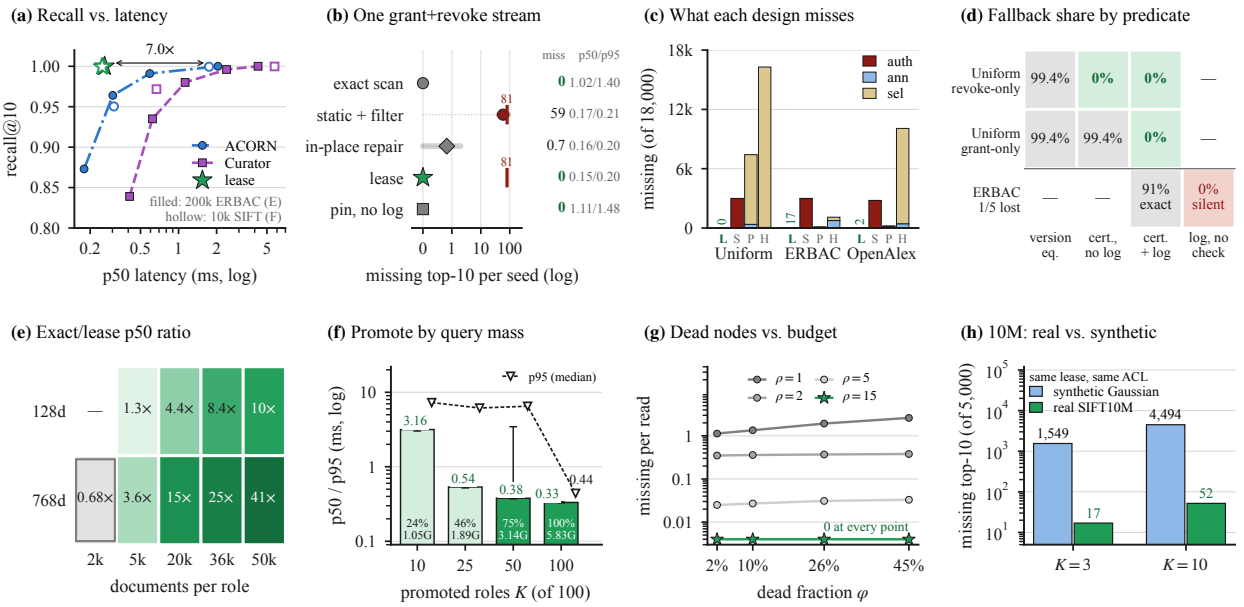


Fig. 3. The main paper's evaluation figure. (a) Recall@10 against p50 (filled: 200k ERBAC sweep; hollow: one snapshot on the official 10k SIFT queries). (b) Five designs in one C++ kernel on one grant+revoke stream (1M SIFT, three seeds): missing top-10 results per seed (1,150–1,200 reads ×10 returned), log axis, bands span the seeds, 0 drawn at the left edge, p50/p95 in the right column; red ticks mark the unauthorized candidates each arm's own top-k held before the filter. (c) Missing results split into auth/ann/sel (L/S/P/H = lease, static, partition, shared). (d) Fallback share per validity predicate (gray: reads sent to the exact scan; red: answered from an incomplete log). (e) Exact-to-lease p50 ratio by role size and dimension; the outlined cell is the only one where exact wins (128d SIFT: decouple sweep, three runs on a Gold 6330; 768d Wikipedia: crossover sweep, five runs on a 9950X). (f) All-role traffic as the promoted set grows (bars: p50 with min–max; triangles: p95 medians). (g) Dead fraction against candidate multiplier ρ (gray: $\rho \leq 5$; green: ρ=15). (h) At 10M the same lease misses 17 and 52 results on real SIFT10M against 1,549 and 4,494 on the synthetic Gaussian corpus, at K=3 and K=10.

boundary. We measured whether that pays and whether it can stand in for coverage.

a) Protocol: 1M SIFT, the 1,000-role Uniform ACL (5,000 documents per role), 200 queries, ten promoted roles, four readers and one writer. Revoke-only streams of 10k, 20k and 30k changes remove 20%, 40% and 60% of each role's documents (the writer retries an already-revoked pick, so the fractions are exact); grant-only streams of 5k and 10k add 10% and 20%. Three seeds for revocations, two for grants. The stream stops, every role is at its final version, and only then do we run the pass on the quiesced graph: ROE (threshold T=1), then either nothing, or a refill that replaces each pruned edge with the dead node's nearest live neighbours (up to $M/2$ per node), then RDN (components smaller than half the live graph, 2-hop repair, M=8) and RUE. A rebuild from the visible set, same M and `efConstruction`, is the deep reference. Nothing is repaired during the stream: a graph that serves older pins may not be rewired underneath them.

b) The official code: The authors' implementation (`github.com/huijunjin/PRO-HNSW`, built here against pybind11) runs on the same SIFT corpus under its own protocol: 200k vectors, one bulk delete-and-reinsert batch, recall@10 at `ef`=100. Its runner defaults to a single-layer graph (`graph=flat`), which is what these numbers use, three seeds: vanilla HNSW reaches 97.73% at δ=0.2 and 97.69% at δ=0.8, PRO-HNSW 98.69% and 98.61%, and a full rebuild 98.47% and 98.54%; the three modules cost 0.14 s inside a 1.0 s (δ=0.2) or 5.3 s (δ=0.8) update. With `--hierarchical` and one seed the same arms read 97.90/98.62/98.42 and 97.50/98.88/98.42, so the ranking survives the switch but the values are protocol-specific and are not compared with the lease's hierarchical HNSW. That scenario deletes and reinserts vectors, so its numbers are not the table below either; it establishes on our data that the modules help the graph, which is the premise of the adaptation that follows.

c) Structure: Across the 27 revocation runs ROE takes 170,710 obsolete edges to 326, 53 isolated nodes to 0 and 21,981 one-way edges to 373, in 9.2 ms summed over ten roles. The edge counts here are level-0 edges between live nodes, while ROE's 177,266 pruned slots include 6,882 above level 0. At level 0 the accounting closes exactly: 480,359 edges before, minus 170,384 pruned, plus 116,027 from the refill, 294 from RDN and 88,943 from RUE, gives the 515,239 measured after. The pass costs 93.2 ms against 761.6 ms for the rebuild, 12%, but ten times ROE alone (9.2 ms): the refill buys its recall

TABLE IV

MISSING TOP-10 RESULTS AFTER A REVOKE-ONLY STREAM (OF 2,000 SCORED RESULTS, 200 QUERIES, MEAN OF THREE SEEDS; RECALL@10 IS $1 - \text{missing}/2{,}000$). φ IS THE SHARE OF EACH ROLE'S DOCUMENTS REVOKED, ρ THE CANDIDATE MULTIPLIER. *lease* SERVES FROM THE PUBLISHED GRAPH, *+prune* AFTER ROE/RDN/RUE, *+refill* WHEN THE REFILL RUNS BETWEEN ROE AND RUE, *rebuild* FROM THE VISIBLE SET; *in-place* IS THE UNVERSIONED GRAPH, WHICH MARKS DELETIONS AND LETS THE SEARCH SKIP THEM.

φ	ρ	lease	+prune	+refill	rebuild	in-place
20%	1	466	257	141	189	170
20%	3	30	38	**12**	15	15
20%	15	**0**	**0**	**0**	**0**	**0**
40%	1	813	363	173	162	114
40%	3	33	78	16	**12**	8
40%	15	1	3	**0**	**0**	**0**
60%	1	1,186	580	319	145	73
60%	3	102	142	35	13	**4**
60%	15	**0**	5	**0**	**0**	**0**

TABLE V

GRANT-ONLY STREAMS: MISSING TOP-10 RESULTS (2,000 SCORED, TWO SEEDS) AND, IN PARENTHESES, THE AUTHORIZATION TERM. *no log* IS THE REPAIRED GRAPH READ WITHOUT THE GRANT LOG; THE REFILL CHANGES NONE OF THESE CELLS, BECAUSE A GRANT ADDS NO DEAD NODE TO PRUNE.

granted	ρ	lease	+repair	rebuild	no log (auth)
10%	1	183	**170**	195	358 (188)
10%	15	**0**	**0**	**0**	188 (188)
20%	1	158	**147**	190	496 (348)
20%	15	**0**	**0**	**0**	348 (348)

with an order of magnitude of maintenance. Reciprocity after refill is 8,197 one-way edges rather than ROE's 373, because refilled lists leave RUE little room to add the reverse edge; that is cosmetic, not a recall effect.

d) Recall: Pruning alone helps only where the budget is tight enough for dead candidates to displace live ones (ρ=1: 466 to 257 at 20% dead) and hurts at ρ=3 (33 to 78 at 40% dead), because it removes navigation without replacing it. The refill removes that regression, but it does not close the gap to a rebuild in general, and the gap widens with the dead fraction: at 20% it beats the rebuild (141 against 189 at ρ=1, 12 against 15 at ρ=3), at 40% it is behind (173 against 162, 16 against 12), and at 60% it is behind by 2.2× and 2.7× (319 against 145, 35 against 13). Only at ρ=15 do the arms coincide at zero, which is the operating point of the main text and not evidence that repair substitutes for a rebuild. Latency follows: p95 is 0.006 against 0.011 ms at ρ=1 with 40% dead, 0.013 against 0.022 at ρ=3, and 0.050 against 0.072 at ρ=15 with 60% dead. The in-place graph still beats the repaired lease graph at ρ=1 (170/114/73) because it marks deletions and the search skips them—a physical delete the versioned graph gives up. Leaks are zero in every arm of all 70 runs.

e) Coverage: Under grants the repaired graph without the log misses 188 of 2,000 results at 10% granted and 348 at 20%, all of them authorization losses—the granted documents were never in the graph, so no edge rewiring can return them; the log brings both to zero. This is the split the main text claims: repair restores structure, the certificate restores candidates.

f) Scope: The churn measured here is policy churn: documents enter or leave a role's visible set, and the only physical deletion is the in-place arm's mark. Corpus churn—a document re-embedded and reinserted with a new vector—is a different failure, because the lease's materialised vectors go stale and its answer is a new publication, not an edge repair; the numbers above do not apply to it.

g) Reading: Repair is a quality tool with a boundary, not a substitute. It restores what the graph lost and nothing the log has not delivered; at 93 ms it is 12% of a rebuild but ten times ROE alone, and it beats a rebuild only while the dead fraction is low. Whether that trade pays is deployment specific. The certificate, not the repair schedule, is what keeps the answer correct. Raw outputs: `experiments/results/` with `repair_20261109/` and `repair_refill2_20261109/`; the official runs are in the PRO-HNSW tree's `results/phase1/`.

h) Arm names across harnesses: One name does not travel: the arm labelled `lease` in the d34 concurrent-backlog harness (experiments/mechanism/d34_concurrent_arms.py) is the *in-place repair* prototype of §V-E, not the certified kernel. It is a mutable per-role HNSW that applies each change in place and filters at read time; it holds no grant log and no interval, so its $auth$ axis is expected to go positive once the repair rate falls behind the change rate. That is the number the main text quotes for the fast-clock ERBAC stream (3 authorized results for the in-place arm against 639 for the unrepaired static index): it is the motivation for adding the log, not a measurement of the shipped kernel. The certified kernel's arms are *cert.+log* and the three predicate controls of §V-G; every zero-$auth$ claim in this paper comes from those.

X. BASELINE PROTOCOL AND ENVIRONMENT

All three systems run on the same 200k ERBAC ACL, the same query set, one host, and the container's 16-core quota. Two environmental facts matter for reproduction and are easy to get wrong.

a) The quota is smaller than `nproc`.: `/sys/fs/cgroup/cpu.max` reads `1600000 100000`, that is 16 cores, while `nproc` reports 128. OpenMP sizes its pool from `nproc`, so with default settings both baselines oversubscribe eight-fold: ACORN's median goes from 0.30 ms to 300 ms and

Curator's `train` at 50k vectors does not finish in fourteen minutes. Exporting `OMP_NUM_THREADS=16` restores 0.30 ms and 0.93 s. Every number below is measured that way.

b) Driver details.: The ACORN driver links the official fork's `libfaiss.a` and calls the filtered `search` with a per-role bitmap. The Curator driver uses `MultiTenantIndexIVFHierarchical`: `train` once, `add_vector_with_ids`, `grant_access(label, role)` per visible pair (Curator allows a vector to belong to several tenants, which matches an overlapping ACL), then `search` per role. Two details cost us a day: role ids must be stored as 32-bit, and `search_ef` must be greater than zero or the index asserts.

c) HoneyBee.: We also ran the official pipeline (PostgreSQL 16 with pgvector, its 11-step protocol) on the same SIFT-128 corpus at top-100: the role partition answers in 17.0 ms at recall 0.976–0.994 using 129.5 MB, the dynamic partition in 24.1 ms at recall 0.995–1.0 using 140.5 MB, and the per-user combination partition reaches recall 0.80 only at 2,972.8 MB, 23× the role partition's space. This is a static partitioning pipeline, not a streaming authorization trace; the co-access partition in our figures is our own shape baseline.

TABLE VI
BASELINE SWEEP, 200K ERBAC, 100 QUERIES, 16 THREADS, THREE BUILDS EACH (LEASE: FIVE SEEDS). ROWS ARE MEASURED OPERATING POINTS, NOT EQUAL-RECALL POINTS. RECALL IS AGAINST AN EXACT SCAN OF THE SAME VISIBLE SET.

system	setting	p50	p95	recall
lease	ρ=15	**0.260**	**0.322**	**1.000**
ACORN	ef=16	0.179	0.215	0.873
ACORN	ef=32	0.302	0.372	0.964
ACORN	ef=64	0.592	0.750	0.991
ACORN	ef=128	1.037	1.556	0.998
ACORN	ef=256	2.050	2.509	1.000
Curator	se=100	0.285	0.453	0.712
Curator	se=400	0.622	1.010	0.935
Curator	se=1600	2.397	3.137	0.996

XI. MEASUREMENT VARIANCE

Early baseline runs showed occasional p50 of 0.32–0.50 ms and p95 near 5 ms, with `missing` still zero. We traced this to a leftover benchmark process of our own that had been spinning for nine and a half hours on the same quota (a writer deadlock fixed later in the kernel); the other tenant on the host used about one core. After clearing it, five lease builds read 0.260 ± 0.004 ms p50 and 0.322 ± 0.021 ms p95 with no outliers, and the ACORN and Curator rows were re-run in the same quiet window. The lesson is practical: a benchmark script needs a timeout and a cleanup trap, or it contaminates every measurement that follows.

TABLE VII
RECOVERY LIFECYCLE, PRE-FIX VERSUS FIXED KERNEL, SAME BINARY INPUTS (32 DOCUMENTS, ONE ROLE, FOUR QUERIES). A1–A3 USE A SEQUENCER; B1 USES A SOCKET THAT ACCEPTS AND NEVER ANSWERS.

case	pre-fix	fixed
A1 replay, then revoke	missing 2, auth 2, no fallback	0 / 0, no fallback
A2 grant→revoke→grant replay	mismatch 1, visible 8/9	0, visible 9/9
A3 replay, then two changes	missing 2, auth 2	0 / 0
B1 stalled WAL commit	hangs (no socket timeout)	refused, no state written

XII. RECOVERY AND REPLAY

The certified kernel commits every change to the sequencer (`experiments/service/seqd.py`), which appends and fsyncs the record before returning its version. A restarted kernel replays the log from zero, restores the visible set and the per-document transition history, and then *rebuilds* the lease over the replayed set at its last durable epoch: the new interval starts at $a_L = \omega_L =$ that epoch, the ring is empty because every earlier grant is already in the rebuilt graph, and history at or below a_L folds into the base state. Serving begins only after this rebuild. A later commit extends the interval from the rebuilt left endpoint.

The first version of this path replayed only the per-document grant and revoke epochs. Two bugs followed. (i) After replay, the first ordinary change advanced ω_L over a graph and a ring that contained neither the replayed grants nor the rebuilt set, so a read could be certified while $V(r, e) \not\subseteq W(L)$. (ii) A document with history *grant→revoke→grant* kept its older revoke epoch and no `multi`/`hist` entry, so it remained invisible after replay. Bug (ii) is invisible to the harness's own `missing`/`auth` counters because the exact oracle calls the same visibility function; both the oracle and the answer are wrong together. We therefore added `SELFCHECK=1`, which compares `visible(d,pin)` with `cur(d)` for every touched document and reports `state_mismatch` and `visible_total`; a nonzero mismatch fails the run even when missing is zero.

Table VII contrasts the pre-fix kernel and the fixed one on the same fixtures. The fixed kernel passes all 23 checks of `e0_wal_lifecycle.py`, while the pre-fix binary fails A1–A3 and blocks forever on B1.

a) Scale of the rebuild.: On the 1M ERBAC (ten head leases, 772,254 materialized documents, 200 durable events) a cold build takes 43.4 s. With `REPLAY=1` the kernel defers the base graph and builds each lease once from the replayed set, so a restart that replays the log and rebuilds the ten leases takes 42.0 s

wall (40.8 s replay plus rebuild), about $0.97\times$ the cold build, and the process peaks at 1.34 GB, the same as a cold process. Recovery is a restart cost of the same order as a cold build, not a per-read tax: after the rebuild the same 100 queries read at $0.193/0.224$ ms p50/p95 with zero authorization misses and zero leakage.

Fail-closed exits, all exercised by the suite: a configured but unreachable sequencer exits 3 before serving; a failed replay RPC exits 4; a log record for a role the run does not materialize exits 5 rather than serve that role from a stale ACL. A commit that times out is treated as *unknown*, refused and never retried on the same connection, because a late reply would desynchronize the stream. A record that was durable but whose acknowledgement was lost reappears on the next replay; that is the correct at-least-once outcome, not a duplicate grant.

b) Contract arms at 1M.: The seven-arm driver passes 7/7 on 1M ERBAC (100 roles, ten promoted, 772,224 materialized documents, 200 queries, 60 changes), on the shipped kernel source: the certified arm reads $0.228/0.316$ ms p50/p95 with zero leakage and two ann misses; version equality and the no-log arm fall back on all 200 reads (14.1 and 14.2 ms); with one grant in five dropped the certificate falls back on 77 of 200 reads while the ungated arm answers all of them; the re-grant sampler reports zero violations and zero live mismatches over 2,000 changes. In this run the injected hole did not surface as an $auth$ miss—both arms end with the same two ann misses—so the 1M run shows the certificate firing rather than a silent loss; the silent loss is the 200k counterexample.

XIII. Reproducing the main evidence

Table I(B) of the main text is backed by experiments/results/cert_matrix_3seed_20261111/ (six arms, three seeds); its means/sds round to the table values.

1) Counterexample (main text Table I, block A): `e1_auth_miss.cpp`, 200k ACL, 100 queries (93 eligible), ten promoted roles; prints auth miss, fallback, leaks and p50 per predicate.
2) Certificate matrix (main text Table I, block B): `lease_certified.cpp` with `CERT=0/1`, `PRED=1`, `NOLOG=1` and `DROP_GRANT=5`.
3) Coverage and deployment sweeps: `churn_arms.cpp` on `qroles_all` with $K = 10/25/50/100$.
4) Concurrency and merge: `WRITERS=1/2/4` with `MERGE_EVERY` per role.
5) Contract tests: `e0_certified_tests.py` (14/14) and `e0_certified_driver.py` (7/7 at 200k and at 1M).
6) Recovery lifecycle: `e0_wal_lifecycle.py` with the kernel binary (23/23; the pre-fix kernel fails A1–A3 and hangs on B1).

XIV. Known limits of this supplement

- The Python harness numbers are protocol-relative; only the C++ rows are absolute.
- The certified kernel takes sequenced concurrent submissions; a lock-free multi-writer protocol is future work.
- The read service is a version-authority prototype, not the certified read path.
- Curator and ACORN are measured at 200k on one host; we did not tune their build parameters beyond the sweeps reported above, so their build times favour them.

XV. Deployment guidance

A. When not to use a lease

When the policy is static, when the workload is revoke-only and the read path already filters, or when the role is small enough that exact is cheaper, the lease adds cost without adding coverage; when the budget cannot cover the head, demote explicitly and report the fallback fraction. Coverage certification itself is not specific to authorization: it applies to any policy-derived materialization whose additions are cheap to witness and whose removals the read path can filter, with a set predicate instead of a freshness scalar. ACLs are the instance where the asymmetry is sharpest, because a revocation must take effect at the next pin while a grant can wait on the log.

XVI. Detailed limitations

a) Baselines.: The HoneyBee-style baseline is our own co-access partition, so its numbers are a capability lower bound; the official pipeline reports 17–112 ms end-to-end SQL on a different protocol, which we do not compare against our kernel numbers. The Veda lattice is our reimplementation (the authors' code is not public) and we report it as a capability bound: it is complete at every budget we tried, and its latency is set by the leftover exact scan rather than by its nodes. Filtered-DiskANN's official build needs L=400 for recall 0.953 at 2.7 ms on a 98k-document role against 0.186 ms at 0.998 for the lease; we tuned neither its build nor its search and ran one role on one thread on a second host, so we read the gap as indicative. The demote-always arm is our own exact path, a lower bound on what scan-on-read pays.

b) Scale.: Every C++ number is at 1M vectors except the 10M sweep. The 10M reads are coverage-weighted and protocol-relative, and ten leases there need about 59 GB, so memory binds before correctness. The two-interval churn matrix runs on the Gaussian corpus and we use its latency only, since its missing counts are the ann term; the 10M runs are deliberately hard (recall@10 is 0.14) and bound build cost and safety, while SIFT10M at the same scale misses 17 of 5,000 at $K{=}3$ and 51.6 ± 0.5 at full coverage.

XVII. Memory buys coverage: the all-role frontier

On 1M ERBAC with all-role traffic (200 reads per point, K of 100 roles promoted), the lease answers only the reads whose role it materialized, so the frontier trades memory for the share of traffic that stays on the lease rather than for the median alone.

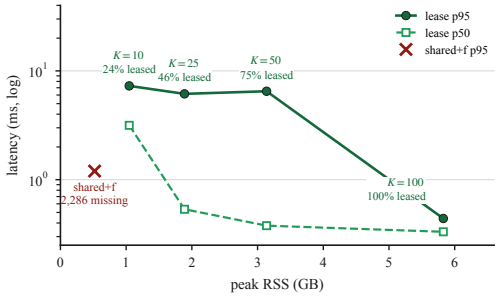


Fig. 4. All-role frontier. Peak RSS is sampled while the run is live; the shared filtered index (×) answers 0.48 ms p50 but misses 2,286 of 5,000 results, the lease misses 0–3 of 2,000 at every K, and its p95 falls only when the promoted set covers the query mass ($K{=}100$, 100% leased).

TABLE VIII
THE SAME FOUR POINTS, MEDIAN AND RANGE OVER THREE RUNS EACH. THE LEASED SHARE IS THE FRACTION OF THE 200 READS THAT A LEASE ANSWERED; THE EARLIER VERSION OF THIS TABLE USED A GLOBAL COUNTER FOR THAT CLASSIFICATION, WHICH MIS-ATTRIBUTED READS BETWEEN THREADS.

K	peak RSS	leased reads	p50 (range)	p95 (range)
10	1.05 GB	24%	3.160 (3.02–3.36)	7.280 (7.20–7.39)
25	1.89 GB	46%	0.535 (0.52–0.83)	6.150 (6.09–7.83)
50	3.14 GB	75%	0.378 (0.37–3.45)	6.500 (5.50–11.12)
100	5.83 GB	100%	0.332 (0.328–0.337)	0.440 (0.431–0.457)

The $K{=}50$ row is not a stable statistic: 150 of its 200 reads take a lease and 50 scan exactly, so its median sits on the boundary between two populations. A rerun of the same configuration on a second host leases the same 150 reads but reads 0.204 ms p50 and 2.89 ms p95, against 0.378 and 6.50 here. The paper quotes the range and takes the full-coverage point ($K{=}100$) as the claim.

TABLE IX
GRAPH BUILD PER PROMOTED ROLE (200K ACL, ONE HOST). THE FIT IS ABOUT 0.09 MS PER DOCUMENT; AT 1 M DOCUMENTS THE FIVE PER-ROLE ARRAYS ADD 18 B PER DOCUMENT EACH.

documents	303	2,401	4,529	6,399	9,874	11,501	14,736
build (ms)	15	156	341	515	844	1,022	1,351

XVIII. Cold-role promotion

A role that has no lease answers from the exact scan. The reference kernel can instead promote it on its first change: it builds a graph over the role's visible set (exact, because the writer refuses any change to a role it has not materialized), records a `promote` event in the shared log, and publishes the state with one atomic store. A restart replays those records first, so the recovered registry matches the one that crashed; `test_promotion_restart.py` asserts that recovery (6/6), and the shipped default still refuses cold-role changes so that a run never materializes a role by accident.

a) Cost of one promotion.: Two independent runs, 120 promotions on the 200k ERBAC ACL, median by role size (16-CPU container on one Xeon 8352V):

b) What it buys, and what it costs.: All 100 roles queried 50 times each (5,000 queries), with the 200-change stream applied before the reads:

c) Attribution: what the interference is not.: A first round of measurements suggested that promoting many roles slows the roles that were already materialized. That reading was wrong: the kernel printed a `WARMUP` value it never read, so the readers started while the writer still had builds to do, and a promotion build is 0.6 s of work inside that window. With the warm-up fixed and the readers released after the last build, five interleaved seeds per arm give:

Bytes do not explain it (the block ballast has the same footprint and no effect) and neither does the shape of the allocation (3,600 small mappings also leave p50 at baseline). Repeating an arm twice in a row shows the same slow mode in *unpromoted* runs (0.590/7.01 ms on the first run of a burst, 0.261/0.555 on its repeat), so the mode is a property of the process or the host, not of promotion.

The decisive control builds every state and then *discards* it (`PROMOTE_DRY`): the same build work, nothing published, the change refused exactly as in the cold path. Five seeds:

arm	p50 (ms)	p95 (ms)	promotions	wall time
refuse (no build)	0.253–0.271	0.30–0.44	0	14–15 s
build and discard	0.341–0.420	1.86–2.35	0	76–90 s
build and publish	0.343–0.369	1.71–1.87	66–76	47–55 s

The discarded builds cost the same as the published ones, so the interference is the *work*, not the

TABLE X

PROMOTION ON THE WHOLE WORKLOAD, NO READER/WRITER OVERLAP. MOVING READS FROM THE EXACT PATH TO THE LEASE PATH CUTS THE OVERALL MEDIAN BY 38% AND THE TAIL BY 35%, AT 2.5× THE RESIDENT SET AND 22 OF 50,000 RESULTS ON THE ANN/SEL AXES (AUTH STAYS ZERO).

arm	lease reads	lease p50	exact reads	exact p50	overall p50	RSS
refuse cold changes	500	0.169	4,500	0.364	0.302–0.311	609 MB
promote on first change	3,300–3,800	0.181	1,200–1,700	0.32–0.40	**0.183–0.193**	1.52–1.58 GB

TABLE XI

ELIMINATING THE EXPLANATIONS FOR THE RESIDUAL TAIL COST (HEAD-ROLE QUERIES, 200 PER RUN, 5 SEEDS, 200K ACL). BOTH CONTROLS MATCH THE PROMOTED ARM'S RESIDENT SET; NEITHER REPRODUCES ITS P50. A SINGLE PROMOTION ALREADY LANDS IN EITHER MODE, SO THE EFFECT IS NOT A FUNCTION OF HOW MANY ROLES ARE PROMOTED.

arm	p50 (ms)	p95 (ms)	RSS
promotion off	0.256–0.267	0.30–3.08	678 MB
promotion on (66–76 roles)	0.347–0.481	1.87–5.72	1.39–1.53 GB
ballast 720 MB, one block	0.250–0.262	0.30–0.38	1.07–1.40 GB
ballast 720 MB, 3,600×200 KB	0.254–0.262	0.30–2.29	1.03–1.42 GB
one promotion	0.253–0.359	0.31–6.08	≈700 MB

materialized state. CPU throttling is not the carrier: the container's cgroup reports `nr_throttled` and `throttled_usec` unchanged across a build-heavy run, so 40–90 s of single-thread work never exhausts the quota. What remains is the page state the build leaves behind, which we have not isolated; the deployment guidance does not depend on it: promote before serving when the workload allows it, and treat a promotion as a maintenance action that costs 0.09 ms per document, not as a free switch. These runs execute in a 16-CPU cgroup on a 128-core Xeon 8352V, so the absolute latencies are container-specific; the ratios and the cost per document are not.

XIX. ONE SNAPSHOT FOR ALL THREE SYSTEMS

The baseline comparison in the main text runs the lease after a 100-change warm-up while the ACORN and Curator drivers read the base ACL statically. Two experiments bound and then remove that difference. Both ran on one 128-core host against the same 200k ERBAC ACL and the same ten head roles; absolute latencies are therefore not comparable to the rows of the main-text table, only ratios are. An earlier recording of this table passed a 500-vector file to drivers invoked with `nq=10000`; Table XII is the re-run on the full official SIFT1M test set (10,000 query vectors, archived as `queries10k_real.f32`). Table XIII puts the conditions (recall, RSS, readers, update state) beside the latency of each row.

TABLE XII

MATCHED SNAPSHOT: THE LEASE'S OWN 100-CHANGE STREAM REPLAYED INTO THE ACORN AND CURATOR DRIVERS, ONE HOST, 10,000 OFFICIAL SIFT QUERY VECTORS, ρ=15 / EF=32, 256 / SE=800, 3200. RECALL IS AGAINST AN EXACT SCAN OF THE SAME POST-CHANGE VISIBLE SET. THE LEASE ROWS USE FOUR READERS EXCEPT THE "ONE READER" ROW; THE BASELINE DRIVERS ARE SINGLE-THREADED.

system	p50 (ms)	p95 (ms)	recall@10	vs lease
lease (100 changes applied)	**0.249**	**0.295**	**0.99954**	—
lease, one reader	0.266	0.324	0.99954	1.07×
ACORN, ef=32, same stream	0.308	0.364	0.95035	1.24×
ACORN, ef=256, same stream	1.742	1.992	0.99943	7.00×
Curator, se=800, same stream	0.960	1.267	0.97177	3.86×
Curator, se=3200, same stream	6.487	7.602	0.99978	26.1×

TABLE XIII

EXECUTION CONDITIONS BEHIND THE MAIN-TEXT BASELINE TABLE (200K ERBAC, ONE HOST, 100 QUERIES). RSS IS PEAK RESIDENT SET FROM THE SAME 200K CAMPAIGN, NOT RE-MEASURED FOR THE MATCHED-SNAPSHOT ROWS; THE LEASE ROWS RUN ON FOUR READERS AFTER A 100-CHANGE WARM-UP, THE BASELINE DRIVERS ARE STATIC AND SINGLE-THREADED. RECALL IS AGAINST AN EXACT SCAN OF THE SAME VISIBLE SET; THE MATCHED-SNAPSHOT REPLAY IS TABLE XII.

system	p50	p95	recall	RSS	readers / state
lease	**0.260**	**0.322**	**1.000**	289 MB	4 / warmed
ACORN ef=32	0.302	0.372	0.964	678 MB	1 / static
ACORN ef=256	2.050	2.509	1.000	678 MB	1 / static
Curator se=800	1.127	1.877	0.980	686 MB	1 / static

a) The lease, static versus warmed.: Same seed set, `changes=0` against `changes=100`: the warm-up moves p50 by 0.8% and p95 by 1.5%, both inside the seed-to-seed spread, and changes 8 of the 200,000 (role, document) memberships. Commands and raw outputs are in experiments/results/ab_static_vs_warm_20261109/.

b) The same stream into all three systems.: The lease now writes the stream it applies (`CHANGES_OUT`), and both baseline drivers apply it to their visible sets before building the ground truth (`CHANGES_IN`). With one post-change snapshot, 10,000 official SIFT query vectors:

The ranking does not move. Curator's se=800 row is its 0.972-recall point; se=3200 reaches the lease's recall at 6.49 ms (26×). The post-change snapshot is within a few percent of the static rows for all three systems on the corrected run, which gives the table's 1.24, 3.86 and 7.00. The same ratios move by up to 15% across reruns on this shared host (§XI); the table reports the speed-matched run. The shift is selectivity, not update processing: ACORN's index is predicate-agnostic and only the per-query bitmap changes, while Curator rebuilds on the post-change ACL. Raw outputs: experiments/results/ab_stream_official10k_20261109/.

c) Reader count, and what the ratios mean.: The main table's lease rows use four readers while the ACORN and Curator drivers are single-threaded. Re-running the same lease on one reader gives 0.266/0.324 ms at the same 0.99954 recall (46 missing of 100,000), 1.07× the four-reader row. The matched one-reader ratios are 1.16× (ACORN ef=32), 3.61× (Curator se=800), 6.56× (ACORN ef=256) and 24.4× (Curator se=3200), against the table's 1.24×, 3.86×, 7.00× and 26.1×. The ordering and the qualitative claim do not change; the four-reader row is 6.7% faster than one reader. The 7× and 26× are delay ratios under the stated search budget and this snapshot, not end-to-end dynamic-maintenance speedups.

d) Returned length in the counterexample.: On 180 constructed queries (200k ERBAC, ten promoted roles, k=10), all four predicates return exactly ten results; the two ungated arms miss the granted Top-1 in all 180, so the failure is a wrong Top-1, not a short list. Raw output: experiments/results/e1_returnlen_20261109/.

XX. Paired significance and reader scaling

a) Paired per-query test.: Every system answers the same 10,000 official queries, so the latency difference can be paired by query. We reran the four systems with per-query timings (`DUMP_LAT=1`) and bootstrapped the median of the per-query difference over 10,000 resamples; a Wilcoxon signed-rank test gives $p < 10^{-300}$ in every pair. The difference is negative when the lease is faster.

TABLE XIV
PAIRED PER-QUERY DIFFERENCES ON THE OFFICIAL 10,000 QUERIES (LEASE MINUS BASELINE). THE PAIRED RUN'S OWN P50S ARE 0.270 MS (LEASE), 0.324 MS (ACORN EF=32), 1.741 MS (EF=256) AND 0.943 MS (CURATOR SE=800); RUN-TO-RUN HOST VARIATION IS WHY THEY DIFFER SLIGHTLY FROM TABLE XII.

baseline	median diff (ms)	95% CI	lease faster
ACORN ef=32	−0.0533	[−0.0542, −0.0523]	89.5%
ACORN ef=256	−1.4623	[−1.4652, −1.4598]	99.9%
Curator se=800	−0.6699	[−0.6732, −0.6673]	99.4%

b) Run-to-run stability.: The paired test above conditions on one run. Three independent runs (separate builds and processes, same inputs, host idle) give:

TABLE XV
THREE INDEPENDENT RUNS ON THE OFFICIAL 10K QUERIES. RATIO RANGE IS AGAINST THE LEASE'S P50 IN THE SAME RUN. RECALLS VARY BY LESS THAN 0.001 FOR ACORN AND ARE IDENTICAL FOR CURATOR AND THE LEASE (46 MISSING OF 100,000 IN EVERY LEASE RUN).

system	p50 mean ± sd (ms)	ratio range
lease	0.250 ± 0.002	—
ACORN ef=32	0.308 ± 0.007	1.20–1.26
ACORN ef=256	1.746 ± 0.033	6.91–7.12
Curator se=800	0.932 ± 0.001	3.70–3.75

The 1.24× point moves by about ±5% across builds, so the paired bootstrap is not the only evidence for the ordering; the cross-run range is reported here. Raw outputs: experiments/results/ab_stream_official10k_20261109/reps/.

c) Reader scaling.: One writer, the same 200k ACL and 10,000 queries on a 16-core quota. QPS here is measured wall-clock with the offline oracle disabled (`SKIP_VERIFY=1`); the oracle is measurement-only and not part of the service. With it enabled, each query is followed by an exact scan for missing/leak accounting, which evicts the graph from cache: the instrumented 10k table reads 0.249 ms, the service-only path 0.199 ms, so the table's ratios are conservative. Throughput scales 13.5× from 1 to 16 readers; p50 rises 10% at 16 from core contention. The baselines are single-threaded: at their p50 their QPS is about 3.2k (ACORN ef=32), 0.57k (ef=256) and 1.0k (Curator se=800), so the lease at one reader is already above ACORN and scales beyond it. A dedicated instrumented single-reader run is 0.266 ms (1.16×/6.56×/3.61×/24.4× against the four baselines).

TABLE XVI
MEASURED SERVICE THROUGHPUT AND LATENCY AGAINST READER COUNT (200K ERBAC, K=10, 100 CHANGES, ρ=15, ORACLE DISABLED).

readers	QPS (wall)	QPS (kernel)	p50 (ms)	p95 (ms)
1	4,970	4,973	0.199	0.239
2	9,730	9,789	0.202	0.244
4	19,890	20,110	0.199	0.240
8	37,570	39,297	0.203	0.244
16	66,980	72,421	0.220	0.275

The two QPS columns are the same runs, not two measurements: *wall* is n_q/read_s (reader-phase wall clock, what this paper quotes) and *kernel* is the `qps=` field, $n_q \cdot \text{threads} / \sum \text{lat}$, which excludes the inter-query gaps and reads 3–8% higher.

XXI. Second host and cold-role boundary

a) Host portability.: The same package, MANIFEST inputs, 100-change stream and 10,000 queries on three AVX-512 hosts: Xeon Platinum 8352V, Ryzen 9950X and Xeon Gold 6330. Streams are byte-identical; absolute latencies differ, so only ratios compare.

TABLE XVII
HOST PORTABILITY OF THE OFFICIAL 10K TABLE (RATIOS AGAINST THE LEASE ON THE SAME HOST). THE 0.95-RECALL ACORN POINT IS THE ONLY COMPARISON THAT MOVES MATERIALLY; THE NEAR-MATCHED AND CURATOR POINTS STAY IN A NARROW BAND.

host	lease	ACORN32	ratio	ef256	ratio	Curator	ratio
8352V	0.249	0.308	1.24×	1.742	7.00×	0.960	3.86×
9950X	0.143	0.144	1.01×	0.874	6.12×	0.469	3.28×
6330	0.212	0.292	1.38×	1.553	7.34×	0.838	3.96×

The ordering holds on all three hosts. The 0.95-recall ACORN gap ranges 1.0–1.4×, the near-matched gap 6.1–7.3× and the Curator se=800 gap 3.3–4.0×. The abstract quotes the near-matched range; §X quotes the 0.95-point range.

b) Is the authorization term invariant to the budget and the graph?: The proposition is algebraic, so we also check it as a measurement: hold $W(L)$ fixed and vary both the candidate budget and the graph, classifying each miss by the definition above rather than by provenance. One 200k-ERBAC role ($|V(r,e)|$=17,056), 100 grants with every fifth dropped from the log (17 dropped, $|W|$=17,039), no revocations, and queries that include each dropped document's own vector, so the hole is exercised. Across nine graph configurations ($M \in \{8,16,32\} \times$ `efConstruction`$\in \{50,100,200\}$) and four budgets ($m \in \{50,150,500,1500\}$) — 36 cells — the authorization term is 17 in every cell, the selection term is 0 in every cell (there are no dead nodes to crowd the budget), and the approximation term falls from 30 to 0. It falls with the budget ($28 \to 0$ at $(8,100)$) and, at a fixed budget, with the graph: at m=50 it is 27, 6 and 3 for $(8,100)$, $(16,100)$ and $(32,200)$. The budget and the graph therefore move the approximation term only, and neither touches the authorization term. SEL=0 is also a property of this construction: there are no revocations and $m \geq k$, so every W-document in the true top-k lies in the exact top-m. The experiment isolates AUTH invariance, not SEL under dead nodes.

c) Why the 0.95-recall gap is host-specific.: A host probe isolates the cause. Single-thread AVX-512 FMA is 92.6 GFLOP/s on the Xeon 8352V and 336.2 on the Ryzen 9950X; triad bandwidth is 6.7 vs 18.2 GB/s; pointer-chase latency is 99.9 vs 95.0 ns—nearly unchanged. The exact AVX-512 scan is throughput-bound and speeds up 2.94× (0.701 vs 0.238 ms for 100 all-role queries); the lease at m=15 speeds up 1.75× (0.250 vs 0.143 ms); ACORN ef=32 speeds up 2.19× (0.317 vs 0.144 ms). The ACORN/lease ratio therefore falls by $2.19/1.75 = 1.25$, which is the observed 1.24→1.01. HNSW is memory-latency-bound, while ACORN's filtered traversal and the exact scan are throughput-bound; the near-matched ef=256 gap falls less (7.05→6.12×) because both sides gain. The 1.24× point is therefore a Xeon-specific measurement, not a portable constant; the Gold 6330 lands at 1.38×, so the 0.95 point moves in both directions. The text quotes the cross-host range 1.0–1.4×.

TABLE XVIII
FIXED $W(L)$, SWEPT BUDGET AND GRAPH (M=16, `efConstruction`=100 SHOWN; THE OTHER EIGHT CONFIGURATIONS GIVE THE SAME AUTHORIZATION AND SELECTION TERMS).

m	missing	auth	ann	sel	p50 (ms)
50	23	17	6	0	1.272
150	17	17	0	0	1.355
500	17	17	0	0	1.480
1500	17	17	0	0	2.192

TABLE XIX
HOST PROBE AND THE PER-SYSTEM SPEEDUP FROM XEON 8352V TO RYZEN 9950X (SAME INPUTS).

metric	Xeon	Ryzen	ratio
FMA (GFLOP/s, 1 thread)	92.6	336.2	3.63×
triad (GB/s, 1 thread)	6.7	18.2	2.72×
pointer chase (ns)	99.9	95.0	1.05×
exact scan p50 (ms)	0.701	0.238	2.94×
lease m=15 p50 (ms)	0.250	0.143	1.75×
ACORN ef=32 p50 (ms)	0.317	0.144	2.19×

d) Protocol control for Curator.: The host table's Curator rows compute the exact truth before each timed query and do not warm up, so the truth scan evicts the graph from cache. A control driver precomputes all truths before any timed search and warms up every query once. The precomputed rows are 0.67/0.30/0.57 ms at se=800 and 5.75/2.06/4.65 ms at se=3200 on the Xeon, Ryzen and Gold hosts. The control driver is shipped as `curator_acl_precomputed.cpp` (sha256 `225f3e96...`); it holds every query's truth in memory, which is why its process RSS reads 1.8 GB against 679 MB for the per-query driver. That difference is the truth buffer, not the index, and the two RSS values are not comparable. The instrumented 26× becomes 23.1× (precomputed Curator over the instrumented lease) or 28.9× (service lease), and the se=800 gap becomes 2.1–2.7× across hosts. The host table's ratios are instrumented; this control is the fairer point.

A. Curator's incremental path

The rebuild comparison below bounds our driver's strategy, not Curator's incremental path. We measure that path on the same 200k ERBAC (role 0, 500-document

TABLE XX
CURATOR PROTOCOL CONTROL ON THREE HOSTS (SAME STREAM, QUERIES AND RECALL). "INSTR" IS THE SHIPPED PER-QUERY-TRUTH DRIVER; "PRE" PRECOMPUTES TRUTHS AND WARMS UP. RATIOS DIVIDE THE PRECOMPUTED CURATOR P50 BY THE LEASE P50 ON THE SAME HOST.

host	lease	in800	pre800	ratio	in3200	pre3200	ratio
Xeon 8352V	0.249	0.960	0.668	2.68×	6.487	5.750	23.1×
Ryzen 9950X	0.143	0.469	0.298	2.09×	—	2.063	14.5×
Gold 6330	0.212	0.838	0.572	2.70×	—	4.652	22.0×

toggle pool, 1,000 queries, nlist=64, nprobe=32, k=10, niter=20). After every checkpoint we run the same queries and recompute recall against an exact scan of the current visible set. A change toggles one pooled document: revoke if present, grant otherwise; updates are single-threaded and timed.

TABLE XXI
CURATOR INCREMENTAL GRANT/REVOKE: UPDATE COST AND QUERY QUALITY AFTER N CHANGES. THE INDEX IS BUILT ONCE; NO REBUILD BETWEEN CHECKPOINTS. RECALL IS AGAINST THE CURRENT VISIBLE SET.

N	upd p50/p95 (μs)	se800 p50	p95	recall	se3200 p50	recall
0	—	0.960	1.240	0.9800	6.891	0.9999
100	1.9/90.3	0.951	1.179	0.9805	7.003	0.9999
1,000	0.89/5.1	0.963	1.191	0.9800	6.984	0.9999
10,000	0.85/4.6	0.907	1.127	0.9800	6.830	0.9999

Ten thousand toggles leave p50, p95 and recall within the spread of this build's checkpoints; across builds the se800 p50 moves by about 20% on this shared host, so the deltas, not the absolute values, are the result. The pool has 500 documents, so every 1,000 changes the ACL returns to its initial state: this is a repeated-toggle stress test, not an evolving ACL. It shows functional stability and update cost, not quality under monotone drift. A single update costs about 1 μs, so the 48 s rebuild is a compaction policy, not a per-change requirement.

a) Concurrent writer.: Four readers and one writer, ten changes per lock, 100 changes total: at se=3200 queries read 6.05/8.57/11.53 ms p50/p95/p99 and the writer's per-batch p99 is 1.54 s; at se=800 queries read 0.677/1.344/2.156 ms and the writer's p99 is 184 ms. Batching raises throughput but does not remove the stall: a batch costs microseconds once it holds the lock, and waits for all four readers to drain. One change per lock at se=100 gives 53.6 ms p99 (§XXIII smoke). Query timing starts after the shared lock; these are search-only latencies, not client end-to-end latency including lock wait. Copy-on-write or writer priority is needed; batching alone is insufficient.

b) End-to-end, incremental (estimate).: The Curator rows above use 1,000 queries and one role; multiplying their p50 by 10,000 gives an estimated 9.1 s at se=800 and 68.3 s at se=3200 plus about 0.01 s of updates, against 54.7/105.5 s for the rebuild variant. These are cost-model estimates, not measured wall-clock totals: the lease's 2.78 s is a different 10,000-query/100-change campaign. The 3.3× and 24.6× are therefore estimates, not a measured end-to-end speedup. The matched-recall *query-only* gap is unchanged: the 26× is a query-path number, not a rebuild artifact.

B. Online churn probe (single role)

This probe fixes every query to one role (role 49), the same role the writer changes; it is not a 10-role query mix. We reran the ACORN online probe after fixing three defects: grants now enter the revoke pool, the truth sample is 500 query IDs drawn uniformly from the 30k (not every 200th read), the writer uses a 30 ms cadence for both search budgets, and all queries target the churn role so every read observes the update stream. A writer-priority flag gives the update thread the lock between reads; without it the writer was starved until the readers finished.

TABLE XXII
ONLINE CHURN PROBE, ONE ROLE (49), 1 READER + 1 WRITER, 50 CHANGES AT 30 MS, 3 SEEDS. ALL SYSTEMS RUN 30K QUERIES (THE 10K SET REPEATED THREE TIMES) AND EVERY CHANGE TARGETS THE QUERIED ROLE. ACORN SEARCH-ONLY AND END-TO-END (BITMAP COPY + ACL LOCK + SEARCH) ARE SHOWN. THE EF=256 $latencyisatiming - onlyrerunwithoutthein - looptruthsample$; ef=32 AND CURATOR STILL CARRY THAT SAMPLE, WHICH COSTS ACORN 12% (0.753 AGAINST 0.669 MS ON THE SAME HOST). ACORN'S RECALL IS EXACT TOP-10 ON 500 RANDOM QUERIES AGAINST EACH PINNED BITMAP, FROM THE SAMPLED RUN. THE LEASE LATENCY IS FROM A `SKIP_VERIFY=1` RUN AND ITS RECALL IS SCORED OFFLINE FROM THE SAME RUN'S RECORDED PINS AND ANSWERS BY FOLDING THE COMMITTED EVENT STREAM (AUTH=LEAKS=0). CURATOR'S TIMER INCLUDES ITS SHARED LOCK, SEARCH AND THE SAMPLE.

system	p50	p95	e2e p50	recall
ACORN-32	0.138–0.139	0.174–0.175	0.141–0.141	0.941–0.954
ACORN-256	0.658–0.669	0.785–0.793	0.661–0.671	0.9990–0.9998
Curator-3200	1.715–1.767	1.991–2.055	same	0.9992–1.0000
lease 1R	0.0565–0.0568	0.0640–0.0643	same	0.99957

All 50 ACORN updates complete inside the read phase; 49–50 of its 30k reads start while an update is in flight. Curator se=3200 reads 1.715–1.767 ms with 48–49/50 updates overlapping (96–97 of 30k reads) and recall 0.9992–1.0000; its writer p99 is 1.59–2.00 ms. At matched high recall the lease is 11.6–11.8× faster than ACORN ef=256 in the timing-only rerun (median $0.669/0.0565 = 11.8\times$) and 30.2–31.3× faster than Curator as measured (median $1.722/0.0565 = 30.5\times$); Curator's timer still carries the in-loop truth sample, so that is an upper bound on the gap. ef=32 is 2.4–2.5× slower but its online recall is 0.941–0.954 against the lease's 0.9996, so that row is not a matched-recall comparison. The in-loop sample costs ACORN 12% on

the same host (0.753 against 0.669 ms), which is why the ef=256 row uses the timing-only run. All rows use 30k queries; the read phase lasts 1.67–1.68 s (lease), 5.2–20 s (ACORN) and 50 s (Curator). For the lease, the first change commits before the reader starts (`WARMUP=1`) and the other 49 commit during the read phase, the last at 1.48 s; no recorded read pins epoch 0. The 30k sequence is the official 10k set repeated three times, so absolute p50 benefits from repeated-query cache; the same sequence is used for every system. This is a same-cadence comparison, not a byte-identical event stream.

a) Where the reads fall in the window.: Because the lease is fast, its reads are mostly *inside* the 1.5 s change window: 26.3k–26.5k of 30k reads start before the last change, and their p50/p95 (0.0565–0.0569/0.0639–0.0643 ms) is indistinguishable from the 3.5k reads after it (0.0566–0.0568/0.0643–0.0645 ms). The baselines are slow enough that only 49–50 (ACORN) and 96–97 (Curator) of 30k reads overlap an update; their headline p50 is therefore mostly a post-window number, and the lease's is mostly an in-window one. That asymmetry does not favour the lease's ratio in any obvious direction, but it is a property of the comparison, not a controlled variable.

b) Same-run recall by event replay.: The performance run records each read's pin and returned documents, and `replay_online_truth.py` folds the committed grant/revoke stream – not the kernel's visibility code – into $V(r, e)$ at each pin and recomputes the exact top-10 with numpy in 64-bit arithmetic (SIFT is integer-valued, so distances are exact; a returned document at the tenth exact distance counts as a hit). On the three seeds the replayed miss is 130/129/129 of 300,000 (recall 0.99957) with zero leaks, on the same run whose latency is reported; the kernel's separate verified run reports 129/300,000. Attribution uses the materialized set, not grant times: the run performs no merge, so G is the role set at build, and a missed result counts as $auth$ only if it is not a node of G. All 130/129/129 misses are nodes of G (graph navigation or budget, not coverage), so $auth$=0. The check is not vacuous: 201/312/235 exact top-10 results are log-only documents, and every one is returned. An earlier lease build promoted ten roles, so only 2 of the 50 changes hit the queried role; the table promotes one role, so all 50 do. (A first version of the script computed distances in 32-bit floats and reported 140/142/146; its rounding misordered documents near rank 10. The shipped records keep both outputs.)

c) Scope.: The probe fixes all queries to role 49, uses 1 reader + 1 writer, 50 changes at 30 ms, a 500-document toggle pool and one host. It is *not* a 10-role workload, not long-term ACL drift, and the three systems do not replay one byte-identical event stream; they share the query set, change count and cadence. The lease latency is service-only (`SKIP_VERIFY=1`); its recall is scored offline from the same run (auth=leaks=0). The 11.6–31.3× ratios above are online ratios for this single-role probe; the main text quotes only the same-snapshot ratios of Table I(F). Curator timing includes its shared lock and search, not client or network time. The matched-snapshot table remains the paper's performance evidence; no claim is made outside this setting.

d) Rebuild-based serial cost.: This is a *rebuild* comparison: our Curator driver reports an ACL change by rebuilding, so the 48 s below is the cost of that driver's compaction strategy. The incremental path is measured in §XXI-A: it costs microseconds and leaves query quality unchanged, so 48 s is a bound, not a per-change requirement. The paper's ratios are query-only. Composing the measured update and query parts for one 100-change batch and 10,000 queries (serial, not a concurrent run) gives the end-to-end picture. The lease pays 0.12 s for 100 changes and 2.66 s of one-reader queries; ACORN's ACL update is a set operation below 1 ms and its queries are 3.08 s (ef=32) or 17.42 s (ef=256); Curator pays a 48 s rebuild plus 6.68 s (se=800) or 57.5 s (se=3200) of precomputed queries. The lease's query-only 1.24× over ACORN ef=32 becomes 1.11× once updates are charged, and the matched 7.0× becomes 6.3×. Curator's 20–38× applies to the rebuild variant; with the incremental path the end-to-end gap is 3.3–24.6× (§XXI-A). It charges Curator's rebuild and no build, including the lease's own 14 s, so the build term is outside every row; amortizing all of them over the same 10,000 queries would add 4.3 ms per query to ACORN and 1.4 ms to the lease. Query-only ratios are therefore conservative for the lease against ACORN, and say nothing about Curator's maintenance mode.

TABLE XXIII
REBUILD-BASED SERIAL COST PER 10,000 QUERIES WITH ONE 100-CHANGE BATCH, FROM MEASURED COMPONENTS. CURATOR'S ROW CHARGES A FULL REBUILD, WHICH IS WHAT OUR DRIVER DOES, NOT AN INCREMENTAL UPDATE.

design	update (s)	query (s)	total (s)	vs lease
lease, 1 reader	0.12	2.66	2.78	—
ACORN ef=32	<0.001	3.08	3.08	1.11×
ACORN ef=256	<0.001	17.42	17.42	6.26×
Curator se=800	48	6.68	54.7	19.7×
Curator se=3200	48	57.5	105.5	37.9×

e) Budget guard.: Algorithm 1 uses the lease only when $\varphi \leq 1 - 1/\rho$ and $\rho k \leq m_{\max}$; otherwise it answers exactly. Without the guard, two regimes trade completeness for latency. At k=1000 on a 20k-document role ($\rho k = 15{,}000 > m_{\max}$), the capped search misses 3 of 50,000 results at 3.54 ms; the guard returns all 50,000 at 2.36 ms. At φ=95% and k=10 the old path misses 192 of 500 results at 0.19 ms; the guard returns all 500 at 0.38 ms. Both regimes keep $auth = 0$ and zero leaks; the

guard moves the failure to exact latency. The published k=10, $\varphi \leq 45\%$ rows never trigger it (m=150 $\leq$ 5000).

TABLE XXIV
BUDGET GUARD ON THE TWO BOUNDARY REGIMES (20K-DOCUMENT ROLE, ONE QUERY SET). "CAPPED" IS THE PRE-GUARD BUDGET; "GUARD" ANSWERS EXACTLY.

setting	missing	p50 (ms)	auth/leaks
k=1000, capped	3/50,000	3.54	0/0
k=1000, guard	0/50,000	2.36	0/0
φ=95%, capped	192/500	0.19	0/0
φ=95%, guard	0/500	0.38	0/0

f) Cold-role boundary.: 200 changes aimed at arbitrary roles (`ANY_ROLE=1`), 100 queries spread over all roles, K=10; the default admission policy refuses a change to a role without a lease, the promotion policy builds it on first change.

TABLE XXV
COLD-ROLE CHURN: ACCEPTED CHANGES, PROMOTIONS AND QUERY LATENCY. REFUSED MEANS THE CHANGE IS NOT APPLIED (NOT SILENTLY IGNORED); BOTH ARMS HAVE ZERO MISSING AND ZERO LEAKS.

mode	accepted	refused	promotions	build (s)	p50	p95
default	25/200	175	0	0	0.482	1.895
promote	200/200	0	74	45.5	0.868	4.132

Promotion accepts every change at about 0.6 s of build per cold role and raises p95 to 4.13 ms while the builds run; the default path keeps p95 at 1.90 ms but leaves 175 changes unapplied. This is an admission boundary, not a certificate failure: neither arm misses a result or leaks.

XXII. CERTIFICATE FAULTS, ACCOUNTING, AND AN INDEPENDENT ORACLE

a) A dropped grant that is the true answer.: The 1M row of the main text's fault table drops every fifth grant, but its three missing results are ann misses on base-ACL documents: in that sample the injected hole does not remove a document the query would otherwise return. A kernel-level script makes the case explicit. Role 0 holds document 0; document 7 is the exact top-1 of query 0; the stream is `grant(7), revoke(7), merge, grant(7) lost`. The merge rebuilds the graph at $a = 2$ without document 7, so at pin 3 the document is authorized ($V(0,3) = \{0,7\}$) but in neither the graph nor the log.

With the certificate on, the read pins 3 outside $[a, \omega] = [2,2]$, scans exactly and returns document 7. With it off, the graph-plus-log path answers from the graph and returns document 0. That is the silent loss the certificate exists to prevent, and it is now a deterministic regression (`script_regression.py`, under `experiments/mechanism/`) rather than a property inferred from a fallback count.

TABLE XXVI
ONE DETERMINISTIC STREAM, FOUR ARMS (16 DOCUMENTS, k=1, FOUR READS). "FALLBACK" IS READS THAT COULD NOT CERTIFY; "TOP-1" IS THE DOCUMENT RETURNED FOR THE QUERY WHOSE TRUE ANSWER IS DOCUMENT 7. THE TWO CONTROL ARMS USE THE SAME STREAM WITHOUT THE LOST LOG ENTRY.

arm	fallback	top-1	missing	$auth$
certificate on, lost grant	4/4	7	0	0
certificate off, lost grant	0/4	0	1	1
certificate on, no loss	0/4	7	0	0
certificate off, no loss	0/4	7	0	0

TABLE XXVII
THE SIX ROWS OF THE MAIN-TEXT FAULT TABLE, RE-RUN WITH THE REPAIRED KERNEL (1M ERBAC, 200 CHANGES, 200 READS, 4R+1W, SEED 0). COUNTS ARE COMPARED; P50 IS NOT, BECAUSE THE HOST WAS SHARED. THE 11-08 RUN HAS 178/200 FALLBACKS IN THE LAST ROW; THE MAIN TABLE REPORTS THE THREE-SEED MEAN 182/200.

predicate	lost	fallback	missing	$auth$
version equality	no	200/200	0	0
cert., no log	no	200/200	0	0
graph+log, no check	no	0	3	0
cert. + log	no	0	3	0
graph+log, no check	1/5	0	3	0
cert. + log	1/5	181/200	0	0

b) Why the old scorer under-counted $auth$.: The kernel attributes a missing result to authorization if the document had no candidate path, i.e. if it is outside $W(L) = V(r, a_L) \cup \{\text{log entries} \leq \text{pin}\}$. The first version tested whether the document's *first* grant predates a_L. A document granted before the build, revoked before it, and re-granted afterwards keeps that first epoch, so the test passed while the document was absent from the rebuilt graph; the miss was then charged to ann. The scorer now tests graph membership, $d \in V(r, a_L)$, through the same history the read path uses, plus the log entries at or below the pin. On the old rule the regression fails exactly the three $auth$ checks (kernel auth 0, oracle 1) and passes 23/23 on the current kernel.

c) The six rows, re-run.: Table XXVII repeats the six rows of the main-text fault table with the repaired kernel on the same 1M ACL. The fallback, missing and $auth$ columns reproduce: version equality and the no-log arm fall back on every read; the two no-check rows and the certified no-fault row miss the same three ann results; the certified hole misses none. The 11-08 run reported 178 of 200 fallbacks for that row; the three-seed re-run in the main table gives 182/200. Reader timing moves the count by a few reads, never the missing or $auth$ counts. Absolute p50 is not compared because the host was shared with other jobs.

d) An oracle that shares no code with the filter.: The kernel scores a read with the same `visible()` predicate that filters its candidates, so a systematic filter

error moves the answer and the reference together. `oracle_check.py` rebuilds $V(r, \text{pin})$ from the base ACL and the committed event stream (`epoch kind role doc`, written by `CHANGES_OUT`) and compares the exact top-k with the documents the kernel actually returned (`QUERIES_OUT`). It shares no kernel code. On the four arms of Table XXVI it reproduces the kernel's leaks, missing and $auth$ counts exactly. On the 1M `nocert_drop5` arm (200 reads) it returns leaks 0 and missing 3, identical to the kernel summary; tied k-th distances are resolved by the same $(distance, id)$ order as the kernel, so the comparison is unique. On a randomized 300-change stream (seed 5) and three 200-change seeds it reproduces missing, and with `SELFCHECK=1` the kernel reports `state_mismatch`=0.

e) A stalled commit does not stall the fallback.: The writer allocates a durable epoch before it takes the read-side state lock, and the exact fallback snapshots the ever-set under its own reader/writer lock. The lifecycle suite drives the case: a lost grant freezes ω at 0, a second change blocks in a sequencer that never acknowledges it (1.5 s client timeout), and the readers start 400 ms into the wait. The fallback completes with p50 0.0009 ms over four reads and the commit is refused after the timeout, so a slow sequencer delays writes, not the exact fallback. The check asserts p50 below 500 ms.

f) Regression totals.: The scripted certificate suite passes 23/23, the WAL lifecycle suite 28/28 (including the stalled commit), the promotion-restart suite 6/6, the sequencer WAL suite 18/18, and the model-level contract suites 14/14 and 9/9 (both interval models). The seven-arm certificate suite at $n{=}10^6$ with 60 changes also passes (7/7; zero leaks in every arm and at most two ann misses; its dropped-grant hole falls back on 77 of 200 reads and version equality on all 200; the main table's 200-change stream freezes the watermark later and falls back on 182 of 200). The 1M numbers in the main text are unchanged by the accounting fix: those runs never merge ($a_L = 0$), and at $a_L = 0$ the old test ($grant_v[d] \le 0$) and graph membership agree on every document—only base-ACL documents pass, and they are in the graph—so $auth$ stays zero under both rules.

XXIII. Mechanism ablation and deployment boundary

a) One controlled kernel, five mechanisms.: The comparison runs in a controlled kernel, not the production kernel: the five arms share one graph implementation, one query set, one deterministic change stream and one scoring routine, and the arm is selected at run time. Only the validity rule, the grant log, the certificate gate and the merge policy change. Its absolute latencies are therefore protocol-relative; the evidence is the difference between arms under one runtime. The version-equality arm uses a per-role version, the most charitable form of the predicate, so its fallback rate is a lower bound. The fault injected into arm C is a lost grant record with a frozen watermark, not a crash; crash recovery is tested separately (SXII). Table XXVIII reruns the five arms with the arm order rotated by query, so no arm always runs first and no arm always pays the cold-cache penalty; the earlier fixed-order run overstated the lease's advantage over the static arm (1.41× fixed order against 1.14× rotated on the same inputs and host).

TABLE XXVIII
Five arms on one kernel, arm order rotated by query (mean±sd of three seeds). *missing* and *pre-leak* are counts per seed over 1,200 reads × 10 results; pre-leak counts unauthorized documents in the arm's unfiltered top-k. Every arm returns zero unauthorized documents. †The version-pin arm falls back to the exact scan once the role has changed.

arm	p50 (ms)	p95 (ms)	missing	pre-leak
exact	1.019±0.015	1.403	0	0
static	0.169±0.000	0.206	59	81
unversioned	0.160±0.001	0.201	0.7	0.3
lease	0.149±0.002	0.201	0	81
pin, no log†	1.108±0.024	1.485	0	0

The fixed-order arms' p99 are 0.112–0.120 ms (grant/revoke); the rotated table's p95 is 0.201. The in-place arm holds a per-role shared mutex, so its tail includes lock wait; the lease read path does not. Update cost is not separately instrumented in this harness.

Arm order changes the latency column by about 20% on this kernel and leaves the counters unchanged, so the counters are the arm comparison and the p50 is a protocol-relative ordering. The *pre-leak* column is the number of unauthorized documents in the arm's own top-k *before* the read-time filter: every arm filters before returning, so the post-filter leak count is zero by construction and cannot distinguish the arms. Pre-leak can: the static arm and the lease arm search the same graph and carry the same 81 unauthorized documents into their candidate lists per 1,200 reads, which is what the filter removes.

b) Five designs in the Python harness.: The same question asked in the Python multitenant harness (1M Uniform, 4R+1W, 200 changes, five seeds, 5,000 returned results per seed, the `arms_20261104` run in the artifact) gives: a stale index with a fresh filter 835±74 missing; the same index without the filter 584 leaked; the incremental lease 0.2 (0–1) missing at 6.11 against 0.83 ms p50; a shared full-corpus index with fresh filtering 4,504±29; and a co-access partition 2,014±75 at 4.60× memory. All but the unfiltered static arm return zero unauthorized documents. The lease here repairs a per-role `hnswlib` index on every change and runs in Python, which is why its p50 is 7.4× the filtered static

TABLE XXIX
MERGE: 2K VS 10K CHANGES (HEAD-10, 50/50 STREAM, 3 SEEDS). THE 2K E ROWS ARE MEASURED WITH THE SAME CADENCE (LONG2K). RSS IS AN UPPER BOUND: THE HARNESS DOES NOT RECLAIM OLD LEASES, SO E INCLUDES STALE GRAPHS.

policy	merges	update	p95 @2k	p95 @10k	RSS
D (no merge)	0	0.01 s	0.116	0.136	698 MB
E (N_M=500)	16	2.43 s	0.116	0.125	799 MB
E (N_M=200)	46	6.88 s	0.117	0.116	985 MB

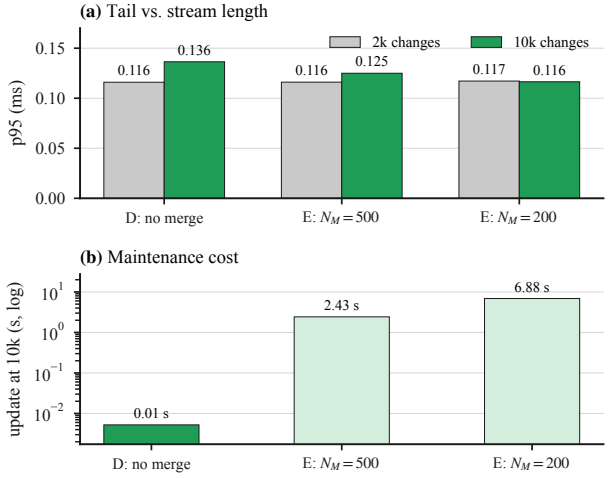


Fig. 5. Merge. (a) p95 at 2k and 10k changes. (b) Update cost of the 10k-change stream. The 2k merge rows are measured with the same cadence; RSS is an upper bound (superseded leases are not reclaimed).

TABLE XXX
THE P95 CLIFF. SAME K=100 BUILD, THREE QUERY MIXES WHOSE TOP-100 ROLES CARRY 90%, 95% AND 99% OF THE QUERIES; P95 STAYS IN THE EXACT POPULATION UNTIL THE FALLBACK SHARE DROPS BELOW 5%; T_F IS THE EXACT-PATH MEAN.

top-100 mass	p_L	p50	p95	T_F
90%	0.899	0.119	0.304	0.304
95%	0.950	0.119	0.280	0.304
99%	0.992	0.118	0.132	0.303

arm's; the C++ arms in Table XXVIII run the same comparison at native speed and are the numbers the main text quotes for Fig. 3(b), with these Python layouts cited separately for the multi-index question.

c) What the merge arm does and does not pay.: Merge keeps *auth* at zero, but it folds grant candidates from the exact log into the approximate graph: three of the nine long-run and 4-core merge runs report 64, 141 and 253 *ann* misses (0.017% of returned results), never an *auth* miss. Its resident set is an upper bound because the harness retains superseded leases and does not truncate the ring; a production reclamation pass would leave the merge arm at roughly the unmerged footprint plus one or two graphs. The additive maintenance term C_{maint}/N_q is a serialized-deployment bound, not an observed read-path cost: on a 4-core budget the merge arm's update time doubles but its p50/p95 stay at $0.102/0.119$ ms, so the term over-predicts the mean (~ 0.20 ms predicted against 0.114 ms measured).

d) Model and break-even.: Across the 30 cells, one global lease mean and an independent exact reference predict each cell's mean within 6.8% (max 13.8%); the mean-cost break-even coverage p_L^* has median 0.12% (p90 0.23%). The p50 rule holds in 27/30 cells (the three misses are at $p_L = 0.4975$) and the p95 rule in 30/30.

e) Scope.: The controlled comparison is at 1M vectors, 5k-document roles, k=10, on one 32-core host, with three seeds per cell; the seed-to-seed standard deviation of p50/p95 is at most 0.004 ms. We do not extrapolate the deltas to 10M or 768d, where only the production kernel is measured, and the reference implementation has a single writer. The HoneyBee numbers are our run of its official static pipeline (SX); the co-access partition in the figures is our own shape baseline, not a HoneyBee reproduction. The crash-recovery suite passes 23/23 (Recovery regression below).

f) Recovery regression.: The replay path was rerun on the current kernel after a bounds check for log entries that name a role outside the ACL (a malformed or stale log). A commit that fails in write or fsync does not consume a version: the sequencer freezes and refuses further commits until replay decides the log position, so a retry cannot create a version gap. The lifecycle suite passes 23/23, including a stalled sequencer that must refuse commits without hanging and an unpromoted role in the log that must exit rather than serve a stale ACL; the promotion-restart suite passes 6/6, the sequencer WAL suite 18/18 (including injected write and fsync failures), and the model-level contract suites contract suites 14/14 and 9/9 (both interval models).

g) A signature-greedy lattice baseline.: Veda (main-text reference [2]) lays out an access lattice over role combinations and plans each query across its nodes. We build a deliberately simpler, Veda-inspired baseline and *not* the authors' planner: exclusive blocks by role signature, greedy node selection by value per byte under a storage budget, one HNSW per selected node, and a scalar exact scan over the leftover; it has no copy/merge planner and no multi-node query plan. It is near-complete at every budget we tried (0 missing at 1–10% of the corpus, 1 at 20% and 6 at 80%), but latency is set by the leftover scan: p50 falls from 23.2 to 19.6 ms as the budget grows from 1% to 20%, and only reaches 8.9 ms at 80%, where the ten head roles are nearly materialized. That last point is the lease's operating point (0.80×) without churn repair, at 0.22 ms: lattice and lease converge on the same materialization,

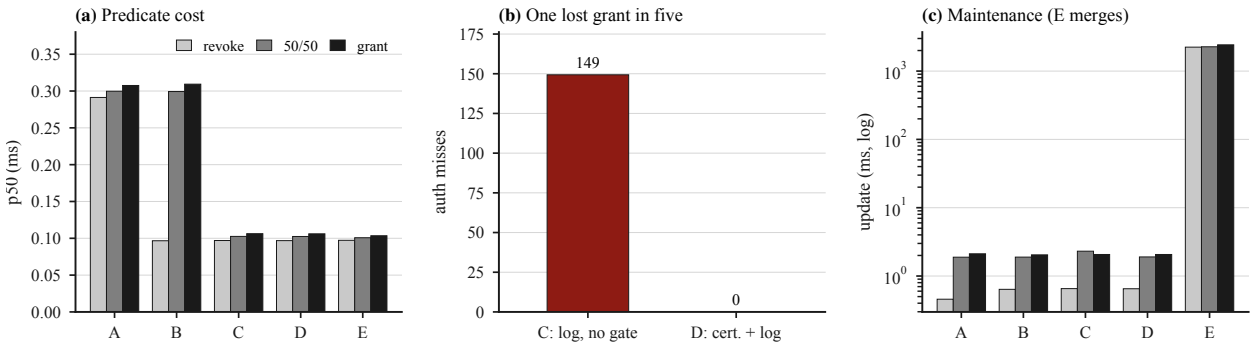


Fig. 6. The five arms on one kernel. (a) p50 per arm under revoke-only, 50/50 and grant-only streams. (b) *auth* misses when one grant in five is dropped from the log (50/50 stream): the ungated arm C returns 149 silently, the certified arm D none. (c) Update cost per stream; E pays for its merges. Means of three seeds; see the table above.

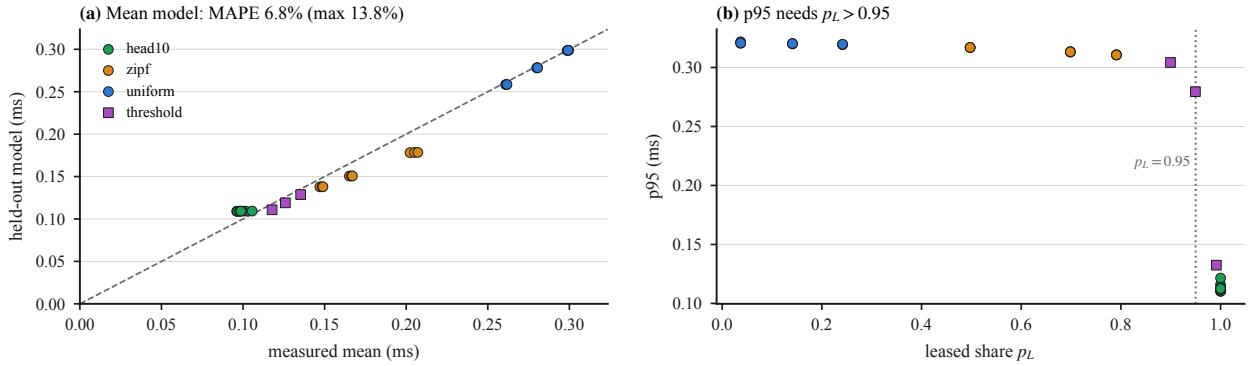


Fig. 7. Held-out mean model and the p95 coverage condition. (a) One global lease mean and an independent exact reference predict each cell's mean within 6.8% (max 13.8%). (b) p95 stays in the exact population until $p_L > 0.95$; the three square points are the threshold cells ($p_L = 0.899, 0.950, 0.992$).

and the lease adds maintenance under churn.

h) Epoch-straddle variant.: A variant that reads the per-role epoch before and after each search is within 3% on the median and indistinguishable on the tail (paired t-test p=0.03 on p50, p=0.54 on p95), but its reads straddle 1–7 change windows per run, a state the per-document test can classify; both arms leak zero.

i) Additional concurrency measurements.: A same-host sweep of the change rate (0, 100, 500, 2,000 and 10,000 events per second, two to three seeds each) leaves p50 at 0.182–0.184 ms and p95 at 0.223–0.229 ms with at most one missing result. Two writer threads applying the same 200-change protocol at the same aggregate rate read within 2% of the single-writer runs in p50 and p95, with zero leaks, straddled reads, retries, fallbacks and pin violations in all six runs. Raw outputs are in `experiments/boundary/results/` and experiments/results/multiwriter_20261107/.